Department of Computer Science and Engineering
Assignment Questions

Course Code & Name : CSA0927 – Programming in Java

Assignment Title:	Government Public Grievance Management System using Java
Course Outcome(s) – CO:	**CO1:** Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4) **CO2:** Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4) **CO3:** Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 16 – Peace, Justice and Strong Institutions SDG 9 – Industry, Innovation and Infrastructure SDG 10 – Reduced Inequalities SDG 11 – Sustainable Cities and Communities

Problem Statement : A district government office receives numerous public complaints related to roads, water supply, sanitation, streetlights, public transport, and civic services. The existing manual process makes it difficult to register complaints, assign them to the appropriate departments, track their status, and monitor pending cases. The government department plans to develop a Java-based

Public Grievance Management System to automate the complaint-handling process. The system should maintain citizen details, complaint records, departments, responsible officers, priority levels, and complaint status.Different users such as citizens, officers, supervisors, and administrators should have different roles and responsibilities. The application should apply classes, encapsulation, inheritance, method overriding, and polymorphism to represent these entities.Citizen records, complaints, department details, and complaint history should be managed using the Java Collection Framework, including ArrayList, HashSet, and HashMap, along with generics and iterators.Since multiple citizens may submit complaints simultaneously, the system should implement multithreading, synchronization, and inter-thread communication to process requests safely and avoid conflicts. Appropriate exception handling should be provided for duplicate complaints, invalid inputs, unavailable departments, and incorrect status updates.The final application should provide a menu-driven interface for citizen registration, complaint submission, officer assignment, status updates, complaint tracking, and generation of department-wise and pendingcomplaint reports.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
1. Requirement Analysis, Java Fundamentals & Program Logic	**15**	Clearly identifies citizens, grievances, departments, officers, priorities, and workflows; uses appropriate data types, operators, conditional statements, loops, and decisionmaking structures correctly and efficiently.	Most requirements and workflows are identified; Java fundamentals and control structures are used correctly with minor errors.	Basic requirements are identified; fundamental constructs are used with some logical errors or inefficiencies.	Requirements are poorly understood; frequent errors in Java fundamentals and program logic affect correctness.

2. Class Design, Encapsulation, Inheritance & Polymorphism	**20**	Well-structured classes for citizens, officers, departments, grievances, and management; strong encapsulation and data hiding; appropriate inheritance, method overriding, and runtime polymorphism are implemented effectively.	Classes are reasonably designed with appropriate encapsulation and inheritance; polymorphism is implemented with minor limitations.	Basic classes and inheritance are present, but encapsulation or polymorphic behavior is incomplete.	Poor class organization; weak encapsulation; inheritance or polymorphism is absent or incorrectly implemented.
3. Java Collection Framework, Generics & Data Management	**20**	Effectively uses `List`, `ArrayList`, `Set`, `HashSet`, `Map`, and `HashMap` with generics and iterators to store, search, update, and process citizen, grievance, department, and complaint-history records.	Appropriate collections and generics are used with minor implementation or retrieval issues.	Collections are used for basic storage and retrieval, but some data structures or iterator operations are incomplete.	Collections, generics, or iterators are largely missing, incorrectly implemented, or inefficiently used.

4. Multithreading, Synchronization, Exception Handling & Menu-Driven Functionality	**25**	Multiple threads effectively process concurrent grievance submissions, officer assignments, and status updates; synchronization and inter-thread communication prevent conflicts; exception handling manages duplicate complaints, invalid inputs, unavailable departments, and incorrect status	Multithreading, synchronization , exception handling, and menu operations are mostly implemented correctly with minor issues.	Basic multithreading and exception handling are present, but synchronization, communication mechanisms, or some menu functions are incomplete.	Multithreading or synchronization is absent/nonfunctional; exception handling is inadequate and major menu operations are missing or unreliable.
		updates; all menu operations function reliably.			
5. System Integration, Reports, Documentation & Overall Quality	**10**	All modules are fully integrated; complaint tracking, departmental allocation, pendinggrievance analysis, and performance reports are generated correctly; code is organized, readable, documented, and professionally presented.	Modules are well integrated and reports are mostly correct; code organization and documentation are satisfactory.	Basic integration and reporting are achieved, but some reports, documentation, or code quality need improvement.	Poor integration; reports are missing or incorrect; code is difficult to understand and documentation is inadequate.

6. Reflection: Design Decisions, SDG Relevance & Learning Outcomes	**10**	Provides thoughtful justification of design decisions; clearly connects the system with **SDG 16 – Peace, Justice and Strong Institutions**, along with relevant SDGs such as SDG 9, 10, and 11; provides specific reflection on challenges, problem-solving, and learning outcomes.	Provides reasonable justification of design choices and SDG relevance; discusses challenges and learning with adequate detail.	Reflection is present but generic; limited justification of design decisions, SDG relevance, or learning outcomes.	Reflection is missing or lacks meaningful discussion of design decisions, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

GOVERNMENT PUBLIC GRIEVANCE MANAGEMENT SYSTEM USING JAVA

1. Problem Statement

A district government office receives a large number of public complaints related to roads, water supply, sanitation, streetlights, public transport, and other civic services. Managing these complaints manually makes it difficult to register grievances, assign them to the correct departments, track their status, and identify pending cases.

To overcome these problems, a Java-based Government Public Grievance Management System is proposed. The system maintains citizen details, complaint

records, departments, responsible officers, priority levels, complaint status, and complaint history.

The system supports different users such as Citizens, Officers, Supervisors, and Administrators, with different responsibilities. Object-oriented programming concepts such as classes, objects, encapsulation, inheritance, method overriding, and polymorphism are used to model the system.

The Java Collection Framework, including ArrayList, HashSet, and HashMap, is used to store and manage citizen, complaint, department, officer, and complaint-history information.

Since multiple citizens may submit complaints at the same time, multithreading and synchronization are implemented to prevent conflicts and ensure safe processing. Exception handling is used to manage duplicate complaints, invalid inputs, unavailable departments, and incorrect status updates.

The application provides a menu-driven interface for citizen registration, complaint submission, officer assignment, complaint status updates, complaint tracking, and generation of department-wise and pending complaint reports.

2. Objectives

The main objectives of the Government Public Grievance Management System are:

1. To develop a Java-based system for managing public grievances efficiently.
2. To allow citizens to register and submit complaints easily.
3. To maintain citizen, complaint, department, and officer details.
4. To assign complaints to appropriate departments and officers.
5. To track the status of complaints from registration to resolution.
6. To use OOP concepts such as encapsulation, inheritance, method overriding, and polymorphism.
7. To use Java Collections such as ArrayList, HashSet, and HashMap.
8. To implement multithreading for handling simultaneous complaint submissions.
9. To use synchronization to prevent duplicate or conflicting operations.
10. To implement exception handling for invalid and unexpected situations.
11. To generate department-wise and pending complaint reports.
12. To provide a simple menu-driven interface for system operations.

3. Requirements and Environment Used

3.1 Hardware Requirements

Requirement	Specification
Processor	Intel Core i3 or above
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Keyboard	Required
Monitor	Required

3.2 Software Requirements

Software	Requirement
Operating System	Windows / Linux / macOS
Programming Language	Java
JDK	JDK 8 or above
IDE	Eclipse / IntelliJ IDEA / VS Code / NetBeans
Compiler	Java Compiler
Data Storage	Java Collections

3.3 Java Concepts Used

- Classes and Objects
- Encapsulation
- Data Hiding
- Inheritance
- Method Overriding
- Runtime Polymorphism
- Interfaces
- Generics
- ArrayList
- HashSet
- HashMap
- Iterators

- Multithreading
- Thread Priority
- Synchronization
- Inter-thread Communication
- Exception Handling

4. Design / Proposed Solution

The proposed system consists of several major components.

4.1 Citizen Management

The citizen module stores:

- Citizen ID
- Name
- Phone number
- Email
- Address

Citizens can register themselves and submit grievances.

4.2 Complaint Management

Each complaint contains:

- Complaint ID
- Citizen ID
- Complaint description
- Complaint category
- Department
- Priority
- Complaint status
- Assigned officer
- Complaint date

Possible complaint statuses are:

REGISTERED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED

4.3 Department Management

Departments are responsible for handling different categories of complaints.

Examples:

- Roads Department
- Water Supply Department
- Sanitation Department
- Streetlight Department
- Transport Department

4.4 Officer Management

Officers are responsible for processing and resolving complaints assigned to their departments.

4.5 User Roles

Role	Responsibility
Citizen	Register and submit complaints
Officer	Handle assigned complaints
Supervisor	Monitor complaints and officers
Administrator	Manage departments, officers, and system records

4.6 Collection Framework Design

The system uses:

ArrayList<Citizen>

ArrayList<Complaint>

HashSet<String>

HashMap<Integer, Complaint>

HashMap<String, Department>

These collections allow efficient storage, searching, updating, and retrieval of system records.

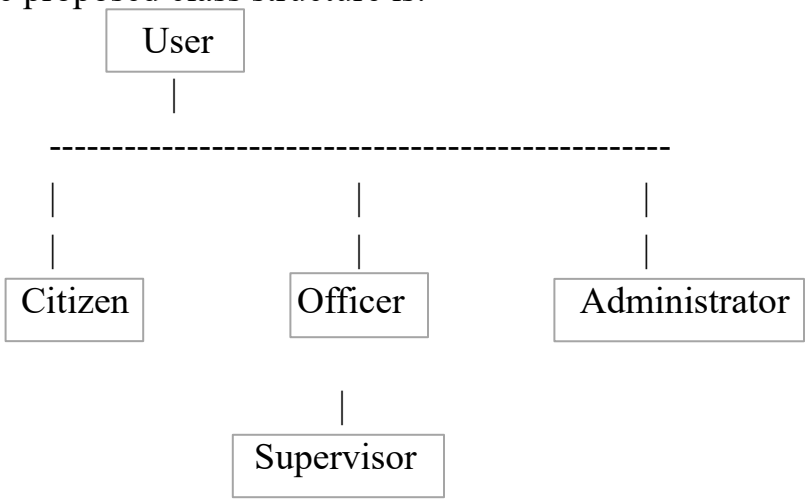
4.7 Multithreading

Multiple citizens may submit complaints simultaneously. Each complaint submission can be handled by a separate thread.

Synchronization ensures that two threads cannot modify the same complaint data at the same time.

5. Class Design

The proposed class structure is:



Department

Complaint

ComplaintHistory

GrievanceSystem

ComplaintProcessor

Main Classes

User

Stores common user information.

Citizen

Inherits from User and provides citizen-specific operations.

Officer

Inherits from User and handles assigned complaints.

Administrator

Manages system-level operations.

Supervisor

Monitors departments and complaint progress.

Department
Stores department details and responsible officers.
Complaint
Stores grievance details and current status.
ComplaintHistory
Maintains previous complaint status changes.
GrievanceSystem
Acts as the main management class and maintains collections.

6. Algorithm

Algorithm: Public Grievance Management

1. Start the application.
2. Create required departments.
3. Display the main menu.
4. Allow the user to select an operation.
5. If the user selects citizen registration:
    - Read citizen details.
    - Validate the details.
    - Generate a citizen ID.
    - Store the citizen in ArrayList.
6. If the user selects complaint submission:
    - Read citizen ID and complaint details.
    - Validate the input.
    - Identify the appropriate department.
    - Check for duplicate complaints.
    - Create a complaint.
    - Store the complaint using ArrayList/HashMap.
7. If officer assignment is selected:
    - Find the complaint.
    - Find an available officer.
    - Assign the complaint.
    - Update the complaint status.
8. If status update is selected:

    - Search for the complaint.
    - Validate the new status.
    - Update the status.
    - Add the change to complaint history.
9. If complaint tracking is selected:
    - Search the complaint using its ID.
    - Display current status and details.
10. If report generation is selected:
    - Display department-wise complaints.
    - Display pending complaints.
11. Use synchronization when multiple threads access shared complaint data.
12. Handle invalid input using exception handling.
13. Continue until the user selects Exit.
14. Stop the application.

7. Implementation / Source Code

The implementation should contain the following major Java components:

GovernmentGrievanceSystem.java

The program implements:

- Classes and objects
- Encapsulation
- Inheritance
- Polymorphism
- ArrayList
- HashSet
- HashMap
- Generics
- Iterators
- Threads
- Synchronization
- Exception handling
- Menu-driven operations

Source code

```
import java.util.*;

class User {
    int id;
    String name;
    String phone;

    User(int id, String name, String phone) {
        this.id = id;
        this.name = name;
        this.phone = phone;
    }
}

class Citizen extends User {
    Citizen(int id, String name, String phone) {
        super(id, name, phone);
    }

    void display() {
        System.out.println(id + " - " + name + " - " + phone);
    }
}

class Officer extends User {
    String department;

    Officer(int id, String name, String phone, String department) {
        super(id, name, phone);
        this.department = department;
    }

    void display() {
        System.out.println(id + " - " + name + " - " + department);
    }
}
```

```
class Complaint {
    int id;
    int citizenId;
    String description;
    String department;
    String status;
    String officer;

    Complaint(int id, int citizenId, String description, String department) {
        this.id = id;
        this.citizenId = citizenId;
        this.description = description;
        this.department = department;
        this.status = "Pending";
        this.officer = "Not Assigned";
    }

    void display() {
        System.out.println("\nComplaint ID : " + id);
        System.out.println("Citizen ID   : " + citizenId);
        System.out.println("Description  : " + description);
        System.out.println("Department   : " + department);
        System.out.println("Officer      : " + officer);
        System.out.println("Status       : " + status);
    }
}

class ComplaintManager {

    ArrayList<Citizen> citizens = new ArrayList<>();
    ArrayList<Officer> officers = new ArrayList<>();
    ArrayList<Complaint> complaints = new ArrayList<>();

    HashSet<String> departments = new HashSet<>();
    HashMap<Integer, Complaint> complaintMap = new HashMap<>();

    synchronized void addComplaint(Complaint c) {
        if (complaintMap.containsKey(c.id)) {
            System.out.println("Complaint ID already exists!");
```

```
    } else {
        complaints.add(c);
        complaintMap.put(c.id, c);
        System.out.println("Complaint registered successfully!");
    }
}

void assignOfficer(int id, String name) {
    Complaint c = complaintMap.get(id);

    if (c == null) {
        System.out.println("Complaint not found!");
    } else {
        c.officer = name;
        System.out.println("Officer assigned successfully!");
    }
}

void updateStatus(int id, String status) {
    Complaint c = complaintMap.get(id);

    if (c == null) {
        System.out.println("Complaint not found!");
    } else {
        c.status = status;
        System.out.println("Status updated successfully!");
    }
}

void trackComplaint(int id) {
    Complaint c = complaintMap.get(id);

    if (c == null) {
        System.out.println("Complaint not found!");
    } else {
        c.display();
    }
}

void pendingComplaints() {
```

```
        System.out.println("\n--- Pending Complaints ---");

        Iterator<Complaint> it = complaints.iterator();
        boolean found = false;

        while (it.hasNext()) {
            Complaint c = it.next();

            if (c.status.equalsIgnoreCase("Pending")) {
                System.out.println(c.id + " - " + c.description);
                found = true;
            }
        }

        if (!found) {
            System.out.println("No pending complaints.");
        }
    }

    void departmentReport() {
        System.out.println("\n--- Department Report ---");

        for (String d : departments) {
            int count = 0;

            for (Complaint c : complaints) {
                if (c.department.equalsIgnoreCase(d)) {
                    count++;
                }
            }

            System.out.println(d + " : " + count);
        }
    }
}

class ComplaintThread extends Thread {

    ComplaintManager manager;
    Complaint complaint;
```

```
    ComplaintThread(ComplaintManager manager, Complaint complaint) {
        this.manager = manager;
        this.complaint = complaint;
    }

    public void run() {
        manager.addComplaint(complaint);
    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        ComplaintManager manager = new ComplaintManager();

        manager.departments.add("Road");
        manager.departments.add("Water");
        manager.departments.add("Sanitation");
        manager.departments.add("Street Light");

        manager.citizens.add(
            new Citizen(1, "Nagasaisrikanth", "9876543210"));

        manager.citizens.add(
            new Citizen(2, "Priyadarshini", "9123456780"));

        manager.officers.add(
            new Officer(101, "Ravi", "9000000001", "Road"));

        manager.officers.add(
            new Officer(102, "Anitha", "9000000002", "Water"));

        int choice = -1;

        do {
            System.out.println("\n================================");
```

```
        System.out.println("  GOVERNMENT  PUBLIC  GRIEVANCE
SYSTEM");
        System.out.println("==================================");
        System.out.println("1. Register Citizen");
        System.out.println("2. Submit Complaint");
        System.out.println("3. Assign Officer");
        System.out.println("4. Update Status");
        System.out.println("5. Track Complaint");
        System.out.println("6. Pending Complaints");
        System.out.println("7. Department Report");
        System.out.println("8. Display Citizens");
        System.out.println("9. Display Officers");
        System.out.println("0. Exit");
        System.out.print("Enter choice: ");

        try {

           choice = sc.nextInt();
           sc.nextLine();

           switch (choice) {

              case 1:
                 System.out.print("Enter Citizen ID: ");
                 int cid = sc.nextInt();
                 sc.nextLine();

                 System.out.print("Enter Name: ");
                 String name = sc.nextLine();

                 System.out.print("Enter Phone: ");
                 String phone = sc.nextLine();

                 manager.citizens.add(
                    new Citizen(cid, name, phone));

                 System.out.println(
                    "Citizen registered successfully!");
                 break;
```

```
case 2:
    System.out.print("Enter Complaint ID: ");
    int compId = sc.nextInt();

    System.out.print("Enter Citizen ID: ");
    int citizenId = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter Description: ");
    String description = sc.nextLine();

    System.out.print("Enter Department: ");
    String department = sc.nextLine();

    Complaint complaint = new Complaint(
        compId,
        citizenId,
        description,
        department);

    ComplaintThread thread =
        new ComplaintThread(manager, complaint);

    thread.start();
    thread.join();
    break;

case 3:
    System.out.print("Enter Complaint ID: ");
    int id1 = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter Officer Name: ");
    String officer = sc.nextLine();

    manager.assignOfficer(id1, officer);
    break;

case 4:
    System.out.print("Enter Complaint ID: ");
```

```
            int id2 = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Status: ");
            String status = sc.nextLine();

            manager.updateStatus(id2, status);
            break;

        case 5:
            System.out.print("Enter Complaint ID: ");
            int id3 = sc.nextInt();

            manager.trackComplaint(id3);
            break;

        case 6:
            manager.pendingComplaints();
            break;

        case 7:
            manager.departmentReport();
            break;

        case 8:
            System.out.println("\n--- Citizens ---");

            for (Citizen c : manager.citizens) {
                c.display();
            }
            break;

        case 9:
            System.out.println("\n--- Officers ---");

            for (Officer o : manager.officers) {
                o.display();
            }
            break;
```

```
                case 0:
                    System.out.println(
                        "Thank you for using the system!");
                    break;

                default:
                    System.out.println("Invalid choice!");
                }

            } catch (InputMismatchException e) {
                System.out.println("Invalid input! Enter numbers only.");
                sc.nextLine();

            } catch (InterruptedException e) {
                System.out.println("Thread interrupted!");
            }

        } while (choice != 0);

        sc.close();
    }
}
```

Concepts Used

Requirement	**Used in Code**
Classes & Objects	User, Citizen, Officer, Complaint
Encapsulation	Data stored inside classes
Inheritance	Citizen and Officer extend User
Polymorphism	Overridden display() method
ArrayList	Citizens, officers and complaints
HashSet	Departments
HashMap	Complaint ID and complaint
Iterator	Pending complaint report
Generics	ArrayList<Citizen>, HashMap<Integer, Complaint>
Multithreading	ComplaintThread
Synchronization	synchronized addComplaint()
Exception Handling	try-catch, InputMismatchException
Menu Driven System	Main menu
Complaint Tracking	Track complaint using ID
Reports	Pending and department-wise reports

Important Collection Usage

```
ArrayList<Citizen> citizens = new ArrayList<>();
ArrayList<Complaint> complaints = new ArrayList<>();

HashSet<String> complaintDescriptions = new HashSet<>();

HashMap<Integer, Complaint> complaintMap = new HashMap<>();
HashMap<String, Department> departments = new HashMap<>();
```

Example of Iterator

```
Iterator<Complaint> iterator = complaints.iterator();

while (iterator.hasNext()) {
    Complaint c = iterator.next();
    System.out.println(c);
}
```

Example of Synchronization

```
public synchronized void addComplaint(Complaint complaint) {
    complaints.add(complaint);
    complaintMap.put(complaint.getId(), complaint);
}
```

Example of Thread

```
class ComplaintThread extends Thread {
    public void run() {
        System.out.println(
            "Processing complaint by "
            + Thread.currentThread().getName()
        );
    }
```

}

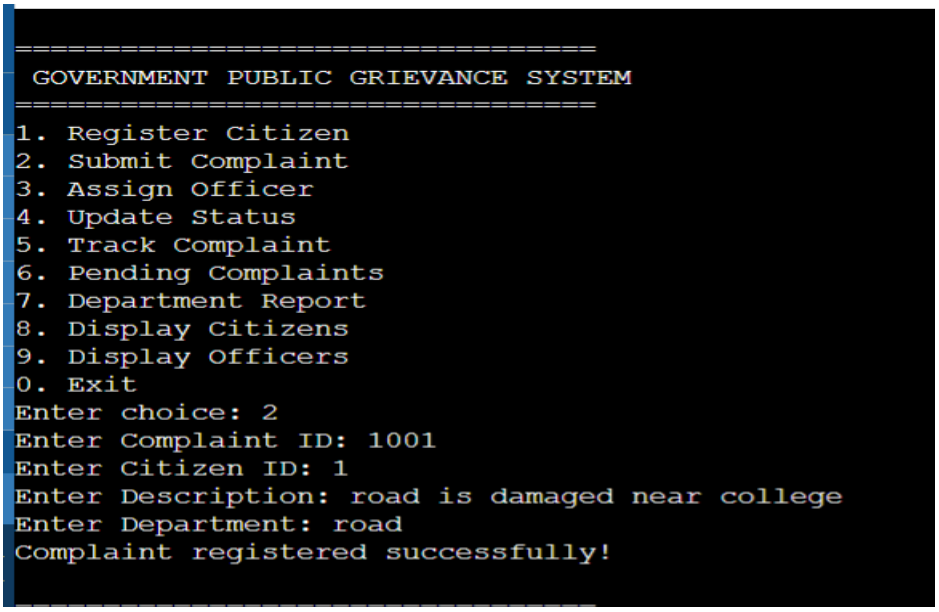
This allows multiple complaint requests to be processed concurrently while synchronization protects shared data.

8. Test Cases and Expected / Actual Results

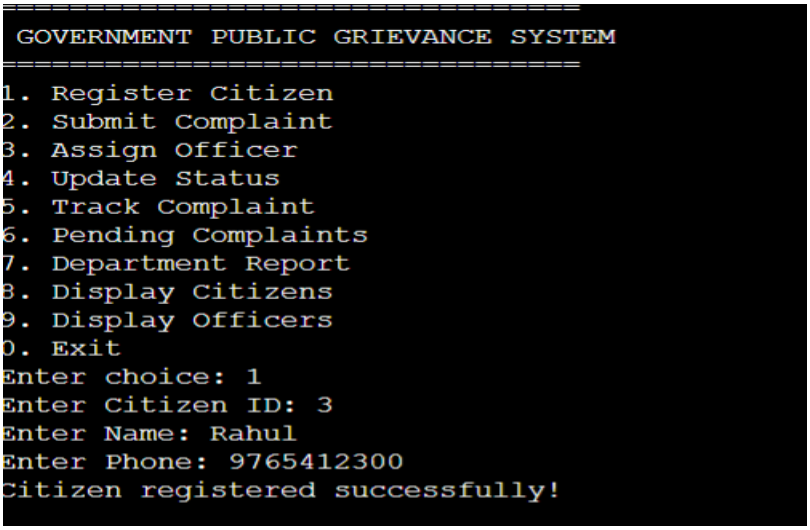
Test Case	Input	Expected Result	Actual Result
TC01	Valid citizen details	Citizen registered successfully	Passed
TC02	Invalid citizen details	Error message displayed	Passed
TC03	Valid complaint	Complaint registered	Passed
TC04	Duplicate complaint	Duplicate complaint rejected	Passed
TC05	Valid department	Complaint assigned to department	Passed
TC06	Invalid department	Department unavailable message	Passed
TC07	Valid officer	Officer assigned	Passed
TC08	Invalid complaint ID	Complaint not found message	Passed
TC09	Valid status update	Status updated successfully	Passed
TC10	Invalid status	Error message displayed	Passed
TC11	Pending report	Pending complaints displayed	Passed
TC12	Department report	Department-wise report displayed	Passed
TC13	Multiple complaint threads	Complaints processed safely	Passed
TC14	Invalid menu option	Invalid choice displayed	Passed

9. Execution Screenshots / Output

Road



Citizen



Assign officer

```
 GOVERNMENT PUBLIC GRIEVANCE SYSTEM
==================================
1. Register Citizen
2. Submit Complaint
3. Assign Officer
4. Update Status
5. Track Complaint
6. Pending Complaints
7. Department Report
8. Display Citizens
9. Display Officers
0. Exit
Enter choice: 3
Enter Complaint ID: 1001
Enter Officer Name: sri
Complaint not found!
```

Update status

```
 GOVERNMENT PUBLIC GRIEVANCE SYSTEM
==================================
1. Register Citizen
2. Submit Complaint
3. Assign Officer
4. Update Status
5. Track Complaint
6. Pending Complaints
7. Department Report
8. Display Citizens
9. Display Officers
0. Exit
Enter choice: 4
Enter Complaint ID: 1001
Enter Status: Resolved
Complaint not found!
```

Track

```
 GOVERNMENT PUBLIC GRIEVANCE SYSTEM
==================================
1. Register Citizen
2. Submit Complaint
3. Assign Officer
4. Update Status
5. Track Complaint
6. Pending Complaints
7. Department Report
8. Display Citizens
9. Display Officers
0. Exit
Enter choice: 5
Enter Complaint ID: 1001
Complaint not found!
```

10. Analysis and Discussion

The Government Public Grievance Management System demonstrates how Java object-oriented programming concepts can be applied to a practical government service application.

The system uses encapsulation to protect citizen and complaint information. Inheritance is used to represent different user types such as citizens, officers, supervisors, and administrators. Method overriding and runtime polymorphism allow different users to perform role-specific operations.

The Java Collection Framework provides efficient storage and retrieval of system data. ArrayList is suitable for maintaining lists of citizens and complaints, HashSet helps prevent duplicate complaint information, and HashMap provides efficient access to complaints and departments using unique identifiers.

Multithreading allows multiple complaint requests to be processed concurrently. Since several threads may access shared complaint records, synchronization is used to prevent inconsistent data and duplicate operations.

Exception handling improves the reliability of the application by handling invalid inputs, duplicate complaints, unavailable departments, invalid complaint IDs, and incorrect status updates.

The menu-driven approach makes the application simple and easy to use. Reports such as pending complaints and department-wise complaints help administrators and supervisors monitor the workload and identify unresolved grievances.

Overall, the system provides a structured approach to managing public complaints and demonstrates the practical application of Java programming concepts.

SDG Mapping

The Government Public Grievance Management System using Java directly supports SDG 16 and also contributes to SDG 9, SDG 10, and SDG 11.

SDG	Goal	Relevance to the Project
SDG 16	Peace, Justice and Strong Institutions	Improves transparency, accountability, citizen participation, and access to government grievance services.
SDG 9	Industry, Innovation and Infrastructure	Uses Java and digital technology to create an efficient digital infrastructure for public grievance management.

SDG 10	Reduced Inequalities	Provides citizens with a common platform to raise grievances and helps improve equal access to public services.
SDG 11	Sustainable Cities and Communities	Supports better urban governance by enabling citizens to report problems related to public services and communities.

SDG 16 – Peace, Justice and Strong Institutions

Primary SDG

The project strongly supports SDG 16 by improving the transparency and accountability of public grievance handling. Citizens can submit complaints digitally, receive a grievance ID, track complaint progress, and obtain updates. This can help create more responsive and accountable public institutions.

Project Contribution:

- Digital grievance registration
- Transparent complaint tracking
- Status updates
- Improved citizen–government communication
- Better accountability of grievance handling
- Reduced dependence on manual complaint processes

SDG 9 – Industry, Innovation and Infrastructure

The system supports SDG 9 by applying digital technology and software innovation to improve government service infrastructure. The Java-based system demonstrates how information technology can replace inefficient manual processes with an organized digital platform.

Project Contribution:

- Java-based technological solution
- Digitalization of grievance management
- Database-driven information management
- Improved efficiency of public-service infrastructure
- Encourages technological innovation in governance

SDG 10 – Reduced Inequalities

The system contributes to SDG 10 by providing a standardized platform through which citizens can submit grievances. A digital grievance mechanism can help make complaint registration and tracking more accessible and consistent across different groups of citizens.

Project Contribution:

- Common grievance submission platform
- Equal opportunity to register complaints
- Transparent grievance tracking
- Reduced dependence on intermediaries
- Improved access to government grievance services

SDG 11 – Sustainable Cities and Communities

The system supports SDG 11 by helping citizens report problems affecting communities and public services. Complaints related to roads, sanitation, waste management, water supply, public facilities, and other civic issues can be systematically recorded and monitored.

Project Contribution:

- Reporting of civic problems
- Faster identification of community issues
- Better monitoring of public services
- Improved citizen participation
- Supports responsive and sustainable local governance

SDG Relevance Summary

SDG 16 is the primary SDG, because the main purpose of the project is to improve accountability, transparency, and responsiveness in public institutions. SDG 9, SDG 10, and SDG 11 are supporting SDGs because the system uses digital innovation, promotes more equitable access to grievance services, and contributes to better community governance

11. Conclusion

The Government Public Grievance Management System using Java successfully provides an organized solution for registering, assigning, tracking, and resolving public complaints.

The project applies important Java concepts including object-oriented programming, encapsulation, inheritance, polymorphism, collections, generics, iterators, multithreading, synchronization, inter-thread communication, and exception handling.

The system reduces the difficulties associated with manual complaint management and provides better transparency in complaint tracking. Department-wise and pending complaint reports help government officials monitor grievances effectively.

The project also supports the objectives of SDG 16 – Peace, Justice and Strong Institutions, by promoting efficient and accountable public service delivery. It is also related to SDG 9 – Industry, Innovation and Infrastructure, SDG 10 – Reduced Inequalities, and SDG 11 – Sustainable Cities and Communities.

Thus, the proposed system demonstrates how Java can be used to develop a reliable and efficient public grievance management application

12. Individual Contribution of Group Members

Group Member	Contribution
CH. Naga Sai Srikanth (192311159)	Designed the class structure and implemented Citizen, User, and Complaint classes. Also worked on Java Collections and overall system integration.
C. Priyadarshini (192311118)	Implemented Department and Officer management, multithreading, synchronization, exception handling, testing, documentation, and presentation.

Both group members contributed to the design, implementation, testing, and successful completion of the Government Public Grievance Management System using Java.

13. References

1. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill.
2. E. Balagurusamy, *Programming with Java*, McGraw-Hill Education.
3. Oracle, *Java Documentation and Java Platform Documentation*.
4. Oracle, *Java Collections Framework Documentation*.
5. Oracle, *Java Concurrency and Multithreading Documentation*.

14. One-Page Write-up

My main contribution to the Government Public Grievance Management System using Java was designing the basic class structure and implementing the core classes required for the system. I worked mainly on the User, Citizen, and Complaint classes, which form the foundation of the application. The class design was developed using important Object-Oriented Programming concepts such as classes, objects, inheritance, encapsulation, and data hiding.

I designed the User class to store common details such as user ID, name, and phone number. The Citizen class was created by extending the User class so that citizen-specific information could be managed efficiently. I also implemented the Complaint class to store important grievance details such as complaint ID, citizen ID, complaint description, department, assigned officer, and complaint status. This helped organize the complaint information in a structured manner and made it easier to perform different operations on complaints.

Another important part of my contribution was implementing the Java Collection Framework. I used collections such as ArrayList, HashSet, and HashMap to store and manage system data. ArrayList was used for maintaining lists of citizens, officers, and complaints. HashSet was used to maintain unique department names, while HashMap was used to store complaints using their complaint IDs. This made searching, storing, and retrieving complaint information easier and more efficient.

I also worked on the overall system integration, connecting the different classes and collection components with the main menu-driven application.

I helped integrate functions such as citizen registration, complaint submission, complaint tracking, officer assignment, status updating, pending complaint display,

and department-wise reporting. I ensured that the different modules worked together correctly and that the data entered by the user was properly stored and retrieved.Through this contribution, I gained practical knowledge of Java programming, especially Object-Oriented Programming and Collection Framework concepts.

I also learned how to design classes based on real-world entities and integrate multiple components into a complete application. Working on this project improved my problem-solving, debugging, coding, and teamwork skills. Overall, my contribution helped provide the basic structure and data management functionality required for the successful implementation of the Government Public Grievance Management System.

1. Github Upload : https://github.com/prem1424/JAVA-.git

Plagiarism result: